Épico 1: Configuração do pipeline (Fase 1)

Histórias de Usuário

Definições e versionamento do codebook

HU-101 - Como Administrador de Projeto, quero cadastrar definições com título, para que a LLM receba a estrutura da tarefa que quero estudar.

- Existe um formulário na aba de configuração da Fase 1 para adicionar definições
- O título é obrigatório, e tentar salvar uma definição sem título mostra o erro no próprio campo
- Uma definição salva aparece na lista imediatamente após o salvamento
- A descrição da definição não é oferecida nesta fase, porque pertence à Fase 2

HU-102 - Como Administrador de Projeto, quero escolher o tipo de cada definição entre Categoria, Dimensão de qualidade e Diretriz, para que a ferramenta e a minha equipe saibam que natureza de tarefa está sendo montada.

- O formulário oferece exatamente esses três tipos, com esses nomes
- O tipo escolhido aparece junto do título na lista de definições
- Cada tipo tem uma explicação curta acessível na própria tela, para quem não conhece o vocabulário

HU-103 - Como Administrador de Projeto, quero que o tipo da nova definição já venha pré-selecionado conforme o tipo de tarefa que declarei ao criar o projeto, para que eu não precise repetir a mesma escolha em cada definição.

- Projeto de Classificação abre a nova definição com Categoria pré-selecionada
- Projeto de Avaliação de qualidade abre com Dimensão de qualidade
- Projeto de Geração abre com Diretriz
- Projeto sem tipo declarado, misto ou outro, abre sem pré-seleção
- A pré-seleção não impede a troca, e nenhum outro comportamento da ferramenta muda por causa dela

HU-104 - Como Administrador de Projeto, quero poder trocar o tipo de uma definição específica, para que um projeto misto não fique preso a um único tipo.

- O tipo é editável em qualquer definição enquanto a versão vigente estiver em aberto
- Definições de tipos diferentes convivem no mesmo codebook sem aviso nem bloqueio

HU-105 - Como Administrador de Projeto, quero registrar uma observação em texto livre sobre a versão que estou salvando, para que eu lembre depois por que mudei o codebook.

- O campo de observação é opcional e some se não for preenchido
- A observação fica presa à versão em que foi escrita e aparece no histórico
- Salvar sem observação é permitido e não gera aviso

HU-106 - Como Administrador de Projeto, quero que o salvamento crie uma nova versão do codebook com numeração sequencial sempre que a versão vigente já estiver congelada, para que eu tenha o histórico completo da evolução das regras.

- O primeiro salvamento do projeto cria a versão 1
- Salvar quando a versão vigente já está congelada cria a versão de número imediatamente superior à maior existente
- Salvar quando a versão vigente está em aberto atualiza a própria versão, sem criar número novo
- Dois salvamentos disparados ao mesmo tempo não produzem duas versões com o mesmo número
- A versão registra quem salvou e quando, e a atualização no lugar registra também a data da última alteração

6a. Como Administrador de Projeto, quero corrigir o codebook enquanto ele não tiver sido usado em nenhuma rodada, para que um erro de digitação não me obrigue a criar uma versão só por causa dele.

- A versão vigente é editável enquanto for a mais recente e não tiver sido usada: dá para alterar título, tipo, ordem e observação, e incluir ou remover definições
- A tela diz que a versão está em aberto e o que vai congelá-la
- Assim que a versão é usada, a edição some da interface e uma chamada direta é recusada
- Versão que não é a mais recente nunca é editável, tenha sido usada ou não
- Como não existem rodadas na Fase 1, nesta fase a versão vigente está sempre em aberto, e a regra fica pronta para quando o Épico 2 criar rodadas

HU-107 - Como Administrador de Projeto, quero que nenhuma versão congelada possa ser apagada ou sobrescrita, para que o histórico da pesquisa seja confiável.

- Não existe ação de apagar versão em lugar nenhum da interface
- Criar uma versão nova deixa o conteúdo da anterior exatamente como estava, incluindo definições, tipos, ordem e observação
- Uma chamada direta que tente alterar versão congelada é recusada
- Congelar é irreversível: não há ação que reabra uma versão já usada

HU-108 - Como Administrador de Projeto, quero ver a lista de versões do codebook com a data e a observação de cada uma, para que eu consiga reconstruir o raciocínio da equipe.

- O histórico lista todas as versões, da mais recente para a mais antiga
- Cada linha mostra número, data, autor e observação, se houver
- Dá para abrir uma versão antiga e ver as definições como estavam nela, em modo leitura
- Fica claro qual é a versão vigente e se ela ainda está em aberto ou já congelou

HU-109 - Como Administrador de Projeto, quero ser impedido de salvar um codebook sem nenhuma definição, para que o projeto não avance com um pipeline vazio.

- Tentar salvar com a lista vazia é recusado, com mensagem dizendo que é preciso ao menos uma definição
- Nenhuma versão é criada quando o salvamento é recusado
- A regra vale também para chamadas feitas fora da interface

HU-110 - Como Administrador de Projeto, quero reordenar as definições, para que a ordem em que aparecem para a LLM e para a equipe faça sentido.

- A ordem é editável enquanto a versão vigente estiver em aberto
- A ordem salva é a ordem em que as definições aparecem na lista e no envio à LLM
- A ordem faz parte do retrato da versão: abrir uma versão antiga mostra a ordem daquela versão

Prompt

HU-111 - Como Administrador de Projeto, quero escrever o texto do prompt que será enviado à LLM, para que o pipeline tenha uma instrução.

- Existe um campo de texto longo para o prompt na aba de configuração da Fase 1
- O texto salvo aparece de volta na tela exatamente como foi escrito
- O texto é o único campo obrigatório do prompt

HU-112 - Como Administrador de Projeto, quero que o texto do prompt preserve quebras de linha e espaçamento exatamente como digitei, para que a formatação que planejei chegue à LLM.

- Quebras de linha, linhas em branco e recuos são preservados na gravação e na exibição
- Texto longo ganha rolagem própria e não estica nem quebra o layout da página
- O que é enviado à LLM é o texto com a mesma formatação, sem normalização

HU-113 - Como Administrador de Projeto, quero que a alteração do texto do prompt gere uma nova versão quando a versão vigente já estiver congelada, para que eu possa comparar rodadas futuras sabendo qual texto produziu qual resposta.

- Salvar com o texto alterado numa versão congelada cria versão nova, numerada em sequência
- Salvar com o texto alterado numa versão em aberto atualiza a própria versão, sem criar número novo
- Salvar sem alterar o texto não cria versão nova
- A versão registra quem salvou e quando, e a atualização no lugar registra a data da última alteração
- Versões congeladas continuam legíveis e intactas

13a. Como Administrador de Projeto, quero corrigir o texto do prompt enquanto ele não tiver sido usado em nenhuma rodada, para que eu possa ajustar a redação durante a configuração sem encher o histórico de versões que ninguém rodou.

- O texto da versão vigente é editável enquanto ela for a mais recente e não tiver sido usada
- A tela diz que a versão está em aberto e o que vai congelá-la
- "Testar o prompt" não congela versão nenhuma, porque o teste não grava nada e nenhum registro aponta para a versão testada
- Assim que a versão é usada, a edição do texto some da interface e uma chamada direta é recusada
- Versão que não é a mais recente nunca tem o texto editável
- Como não existem rodadas na Fase 1, nesta fase a versão vigente está sempre em aberto

HU-114 - Como Administrador de Projeto, quero dar nome, descrição e registro de mudanças ao prompt, para que a equipe entenda a intenção de cada versão.

- Os três campos são opcionais
- Campos vazios não aparecem na tela de quem só lê, sem rótulo órfão
- Os três aparecem no histórico junto da versão a que pertencem

HU-115 - Como Administrador de Projeto, quero editar nome, descrição e registro de mudanças sem gerar uma versão nova, para que eu possa corrigir um texto descritivo sem poluir o histórico.

- Editar qualquer um dos três e salvar não cria versão nova
- A edição vale apenas para a versão mais recente, esteja ela em aberto ou já congelada
- Tentar editar metadado de uma versão que não é a mais recente é recusado, inclusive fora da interface

HU-116 - Como Administrador de Projeto, quero que o botão de salvar fique desabilitado enquanto o texto do prompt estiver vazio, para que eu não crie uma versão inútil por engano.

- Com o campo de texto vazio ou só com espaços, o botão fica desabilitado
- O botão habilita assim que houver conteúdo
- Uma chamada direta com texto vazio é recusada mesmo com o botão contornado

Itens de entrada

HU-117 - Como Administrador de Projeto, quero cadastrar itens de entrada com nome e conteúdo, para que a LLM tenha o que processar.

- Nome e conteúdo são obrigatórios
- O item salvo aparece na lista do projeto imediatamente
- O item pertence ao projeto, e continua disponível quando o projeto mudar de fase

HU-118 - Como Administrador de Projeto, quero colar o conteúdo de um item direto num campo de texto, para que eu não precise criar um arquivo só para isso.

- Existe um campo de texto longo para o conteúdo
- O conteúdo colado é gravado como está, com a formatação preservada

HU-119 - Como Administrador de Projeto, quero subir um arquivo do meu computador em qualquer formato que a ferramenta leia como texto, para que itens longos entrem sem copiar e colar e sem me obrigar a converter tudo para `.txt`.

- O seletor aceita uma lista explícita de formatos de texto: `.txt`, `.md`, `.csv`, `.tsv`, `.json`, `.xml`, `.html`, `.yaml`, `.log` e arquivos de código
- O conteúdo do arquivo aparece no campo, editável antes de salvar
- O que é gravado é o texto, não o arquivo: não há download depois nem gestão de arquivos
- Imagem, vídeo, áudio e qualquer arquivo binário são recusados, com mensagem dizendo quais formatos valem
- PDF e Word não são aceitos nesta fase, e a mensagem diz para exportar o conteúdo como texto
- Arquivo com extensão aceita que não decodifica como texto é recusado do mesmo jeito, porque o que vale é o conteúdo e não o nome do arquivo

HU-120 - Como Administrador de Projeto, quero cadastrar vários itens, para que eu tenha material suficiente para as fases seguintes.

- Não há limite artificial de quantidade de itens no projeto
- Cadastrar um item não obriga a sair da tela nem recarregar a lista à mão

HU-121 - Como Administrador de Projeto, quero ver a lista dos itens já cadastrados no projeto, para que eu saiba o que já tenho.

- A lista mostra nome e uma prévia curta do conteúdo de cada item
- Dá para abrir um item e ler o conteúdo inteiro, com rolagem e formatação preservada
- A lista deixa claro quantos itens existem no projeto

HU-122 - Como Administrador de Projeto, quero editar e remover um item enquanto ele ainda não foi usado em nenhuma rodada, para que eu possa corrigir um erro de digitação.

- Item nunca usado pode ser editado e removido
- Item já usado em rodada não pode ser editado nem removido, e a interface explica o motivo
- A checagem de uso é a mesma que congela versão de codebook e de prompt, e não uma segunda regra parecida escrita em outro lugar
- Como não existem rodadas na Fase 1, nesta fase todos os itens são editáveis, e a regra fica pronta para quando o Épico 2 criar rodadas

HU-123 - Como Administrador de Projeto, quero ser avisado quando um item ultrapassar o limite de tamanho, para que eu descubra o problema no cadastro e não no meio de uma geração.

- São dois limites diferentes: o arquivo escolhido não pode passar de 2 MB, e o conteúdo gravado não pode passar de 50.000 caracteres
- O limite de arquivo é conferido antes da leitura, para que um arquivo enorme não trave o navegador
- O limite de conteúdo é aplicado no formulário, na ação do servidor e como restrição de schema, com o mesmo valor nas três camadas
- Conteúdo acima do limite é recusado com mensagem dizendo o limite e o tamanho atual
- A regra vale igual para conteúdo digitado e para conteúdo vindo de arquivo, porque depois da leitura os dois são a mesma coisa
- O item recusado não é gravado nem parcialmente

Testar o prompt

HU-124 - Como Administrador de Projeto, quero apertar um botão e ver o que a LLM responde para um item que eu escolher, para que eu descubra se o pipeline funciona antes de convidar avaliadores.

- Existe um botão "Testar o prompt" na aba de configuração da Fase 1
- O Administrador escolhe qual item usar no teste
- A saída da LLM aparece na própria tela, com formatação preservada e rolagem
- O botão fica indisponível enquanto não houver prompt, definição e ao menos um item

HU-125 - Como Administrador de Projeto, quero que esse teste não grave nada, para que a base não encha de respostas descartáveis que ninguém vai avaliar.

- Depois de um teste, nenhuma linha nova existe em tabela nenhuma
- O resultado do teste some ao sair da tela, e não há histórico de testes
- Nenhuma proveniência é registrada, porque não há resposta persistida
- O teste não congela a versão do prompt nem a do codebook: sem resposta gravada, não existe registro apontando para elas, e continuar editando é justamente o fluxo desta fase

HU-126 - Como Administrador de Projeto, quero ver um estado de carregamento enquanto a LLM responde, para que eu não fique em dúvida se o clique funcionou nem clique duas vezes.

- O botão mostra estado de carregamento assim que é acionado
- O botão fica desabilitado durante a chamada, e um segundo clique não dispara segunda chamada
- A tela continua utilizável, sem travar, enquanto a resposta não chega

HU-127 - Como Administrador de Projeto, quero ver uma mensagem de erro clara quando a chamada à LLM falhar, para que eu saiba se o problema é a chave, o modelo ou o tamanho do item.

- Falha de autenticação, modelo inválido, item grande demais e indisponibilidade do provedor produzem mensagens distinguíveis entre si
- A mensagem aparece na tela, sem derrubar a página e sem exigir recarga
- Nenhuma mensagem de erro expõe a chave nem trechos dela
- Depois do erro, dá para corrigir e testar de novo sem sair da tela

HU-128 - Como Administrador de Projeto, quero ver na tela qual modelo está sendo usado, para que eu saiba com o que estou testando.

- O identificador do modelo aparece junto do botão ou do resultado
- O valor exibido é o mesmo que a ferramenta usa na chamada, e vem de configuração, não de texto escrito à mão na tela

HU-129 - Como Administrador de Projeto, quero que o teste use exatamente o que a Fase 2 vai enviar (o prompt mais os títulos das definições), para que o resultado do teste signifique alguma coisa.

- O envio é composto pelo texto do prompt vigente, pelos títulos das definições da versão vigente e pelo conteúdo do item escolhido
- Nenhuma descrição de definição e nenhum critério entra no envio
- A ordem das definições no envio é a ordem salva na versão

Administrador como avaliador

HU-130 - Como Administrador de Projeto, quero me dar acesso de avaliador no meu próprio projeto, para que eu possa avaliar respostas junto com a equipe.

- Existe uma ação explícita para assumir o papel de Avaliador no projeto
- A ação cria um vínculo de avaliador sem mexer no vínculo de administrador
- Quem não é administrador daquele projeto é recusado na ação
- A ação não é oferecida a quem já tem o vínculo de avaliador

HU-131 - Como Administrador de Projeto que virou avaliador, quero passar pelo mesmo consentimento e pelo mesmo questionário de perfil dos demais avaliadores, para que meus dados de perfil não faltem na análise.

- Logo após assumir o papel, o fluxo de consentimento e questionário é apresentado
- Enquanto não concluir, o vínculo de avaliador fica pendente e não dá acesso às telas de avaliação
- O consentimento aceito é registrado com o texto vigente no momento do aceite
- As respostas do questionário ficam vinculadas ao vínculo de avaliador, e não ao de administrador

HU-132 - Como Administrador de Projeto, quero desfazer esse acesso enquanto ainda não enviei nenhuma avaliação, para que eu possa corrigir um clique errado.

- Existe ação de desfazer enquanto não houver avaliação enviada por aquele vínculo
- Depois da primeira avaliação enviada, a ação some e uma tentativa direta é recusada
- Desfazer não afeta o vínculo de administrador nem o acesso ao projeto

HU-133 - Como Administrador de Projeto, quero ver claramente que estou com os dois papéis, para que eu não me confunda sobre o que estou fazendo em cada tela.

- A tela do projeto indica que a pessoa acumula administrador e avaliador
- A lista de membros mostra a pessoa uma vez, com os dois papéis, e não duas linhas soltas sem explicação

Avanço de fase

HU-134 - Como Administrador de Projeto, quero ver em que fase o meu projeto está de verdade, para que a barra de fases deixe de ser decorativa.

- A barra exibe a fase lida do projeto, e não um número fixo no código
- Projeto novo nasce na Fase 1
- Mudar a fase do projeto muda o que a barra mostra, sem precisar de deploy nem de ajuste manual
- As quatro fases continuam visíveis na barra, com a atual destacada

HU-135 - Como Administrador de Projeto, quero avançar o projeto da Fase 1 para a Fase 2 quando terminar a configuração, para que a equipe possa começar a escrever os critérios e a avaliar.

- Com definição, texto de prompt e ao menos um item, a ação de avançar fica disponível
- Confirmar o avanço move o projeto para a Fase 2
- Avançar de fase não congela versão nenhuma: quem congela é o uso numa rodada, e na Fase 2 as definições ainda recebem descrição
- Depois do avanço, a aba de configuração da Fase 1 continua acessível em leitura, porque o histórico não some
- Quem não é administrador do projeto é recusado na ação

HU-136 - Como Administrador de Projeto, quero ser impedido de avançar sem pelo menos uma definição, sem texto de prompt ou sem nenhum item, para que a Fase 2 não comece quebrada.

- Faltando qualquer um dos três insumos, a ação de avançar fica indisponível
- Uma chamada direta feita fora da interface também é recusada, com o motivo
- Nenhum valor de métrica participa dessa decisão: a trava é só por insumo faltando

HU-137 - Como Administrador de Projeto, quero ver exatamente o que está faltando quando o avanço estiver bloqueado, para que eu saiba o que fazer em vez de adivinhar.

- A tela lista os insumos pendentes, nomeando cada um
- Cada pendência leva para o lugar onde se resolve
- A lista some conforme os insumos vão sendo cadastrados

HU-138 - Como Administrador de Projeto, quero confirmar o avanço numa caixa de diálogo que explica o que vai acontecer, para que eu não avance sem querer.

- A ação abre um diálogo antes de qualquer mudança
- O texto do diálogo diz o que a Fase 2 vai fazer com o que foi configurado
- Cancelar não muda nada

HU-139 - Como Administrador de Projeto, quero receber uma confirmação visual de que o projeto avançou, para que eu tenha certeza de que a ação deu certo.

- Depois de confirmar, aparece uma notificação de sucesso
- A barra de fases já mostra a Fase 2 quando a notificação aparece
- Um erro no avanço aparece como erro, e a fase não muda

Acesso e autorização

HU-140 - Como Avaliador, quero não ver as telas de configuração da Fase 1, para que a interface não me ofereça o que não é meu papel.

- A aba de configuração não aparece para quem tem só o papel de Avaliador
- Acessar a rota direto pela URL leva a não encontrado ou a redirecionamento, nunca a conteúdo parcial
- Quem não é membro do projeto recebe o mesmo tratamento

HU-141 - Como Avaliador, quero continuar sendo levado ao consentimento e ao questionário de perfil antes de qualquer tela do projeto, para que o fluxo de entrada continue valendo nas telas novas.

- Membro com onboarding pendente é levado ao fluxo de entrada antes de qualquer rota nova deste épico
- O bloqueio é decidido no servidor, e não apenas escondendo links
- Concluído o onboarding, o acesso segue normalmente

HU-142 - Como Administrador de Projeto, quero que as rotas de salvar codebook, salvar prompt, testar o prompt e avançar de fase recusem quem não é administrador do projeto, mesmo que a chamada venha fora da interface.

- Cada ação verifica no servidor que quem chamou é administrador daquele projeto
- A verificação é por projeto: administrador de um projeto não age sobre outro
- A recusa não revela se o projeto existe para quem não tem acesso
- Cada ação deste épico tem teste provando a recusa do Avaliador

Requisitos Não Funcionais

Integridade e rastreabilidade

RNF-101 - Versão **congelada** é imutável. Uma versão de codebook ou de prompt congela no primeiro uso por uma rodada, e também deixa de ser editável quando uma versão mais nova nasce. Enquanto está em aberto, ela é a área de trabalho do Administrador e pode ser alterada no lugar. Depois de congelada, nenhuma rota, ação ou tela altera ou apaga o que ela guarda. A única exceção são os metadados descritivos do prompt na versão mais recente, que por decisão explícita não são versionados.

RNF-102 - A numeração de versões é sequencial e sem buracos por projeto, calculada como a maior existente mais um, e resistente a dois salvamentos simultâneos. A garantia final é uma restrição de unicidade no schema, não só a lógica da aplicação.

RNF-103 - Toda versão registra autor e data de criação, porque o histórico é dado de pesquisa e precisa dizer quem decidiu o quê e quando. Versão editada no lugar registra também a data da última alteração, para que "em aberto" não vire buraco no rastro.

RNF-104 - O congelamento é dado explícito e verificado no servidor, nunca inferido na tela. Existe uma função só que responde se uma versão está em aberto, usada por toda ação de escrita e pela interface, para que a regra não seja escrita duas vezes com resultados diferentes.

Desempenho e integração com a LLM

RNF-105 - A chamada à LLM é assíncrona e a interface mostra estado de carregamento, impedindo duplo disparo e evitando timeout de navegador. Falha de rede ou do provedor não deixa a tela num estado ambíguo.

RNF-106 - Falha na chamada à LLM não grava nada e não deixa estado parcial. Como o teste de prompt já não persiste por decisão de produto, isso é trivial aqui, mas a propriedade precisa continuar valendo quando o Épico 2 introduzir persistência.

RNF-107 - O acesso à LLM fica isolado num módulo de servidor com superfície mínima, para que a troca de provedor não toque as Server Actions e para que os testes rodem sem chamar a OpenAI.

RNF-108 - Novas rotas não introduzem `loading.tsx`. O diagnóstico registrado em `docs/dev/performance.md` mostra que eles combinados com o throttle do React levam a navegação de cerca de 47 ms para cerca de 310 ms. Indicação de carregamento em navegação usa o wrapper de link já existente no projeto.

Segurança e custo

RNF-109 - A chave da OpenAI existe só como variável de ambiente de servidor, nunca em variável exposta ao cliente, e nunca aparece em mensagem de erro, log ou resposta de API.

RNF-110 - Existe um teto de respostas por projeto, configurável por variável de ambiente, para que um defeito futuro em geração em lote não vire fatura.

RNF-111 - A aceitação de arquivo é por lista de permissão, nunca por lista de proibição. Formato fora da lista é recusado mesmo que não seja imagem nem vídeo, e a checagem final é conseguir decodificar o conteúdo como texto, porque extensão e tipo declarado pelo navegador são palpite.

RNF-112 - Toda ação de escrita deste épico verifica no servidor que quem chama é administrador daquele projeto, conforme a ADR 0007. Não há RLS, trigger nem RPC: a autorização é código da aplicação, e cada caminho tem teste.

Usabilidade e interface

RNF-113 - Textos longos, como prompt, conteúdo de item e saída da LLM, preservam a formatação original, ganham rolagem própria e não quebram o layout da página em nenhuma largura de tela.

RNF-114 - Toda transição de estado irreversível, como o avanço de fase, é precedida de confirmação explícita que diz o que vai acontecer.

RNF-115 - Mensagens de erro dizem o que aconteceu e o que fazer, em português, sem código técnico cru e sem termo em inglês que o usuário do projeto não usa.

RNF-116 - Limites de tamanho de campo aparecem nas três camadas com o mesmo valor, como o projeto já faz: dica no formulário, decisão na ação do servidor e restrição no schema como último anteparo. O limite de tamanho de arquivo é a única exceção e vive só no navegador, porque o arquivo é lido ali e nunca chega ao servidor: o que o servidor confere é o limite de caracteres do conteúdo.

RNF-117 - A interface usa o vocabulário do glossário. Projeto e não sala, rodada e não iteração, item de entrada e não dado de teste, definição e não categoria.

Qualidade

RNF-118 - Cada fatia entra com seus testes verdes, na costura de Server Action, mais teste de página onde houver tela nova e teste unitário onde houver função pura. Nenhuma fatia é considerada pronta com a suíte vermelha.

RNF-119 - Nenhum teste chama a OpenAI de verdade.

Decisões de Implementação

Fases

- `projects` ganha a coluna `phase` (inteiro, default 1, com CHECK entre 1 e 4). Hoje a barra recebe o valor fixo `1` no código da página do projeto, e isso sai.
- A Fase 4 está no escopo da ferramenta. Ela repete a avaliação com itens novos e avaliadores novos, funcionando como teste de replicação. Nada dela é construído neste épico, mas a coluna aceita o valor.
- O avanço de fase é ação consciente do Administrador e nunca é travado por métrica, conforme a ADR 0004. Ele é travado por pré-condição estrutural, que é coisa diferente: sem definição, sem texto de prompt ou sem item, a Fase 2 não tem o que fazer. Essa distinção entra na ADR 0004 reescrita.
- O retorno de fase existe apenas de 4 para 3, e fica fora deste épico.

Codebook

- Duas tabelas novas: uma de versões do codebook (projeto, número da versão, observação, autor, data) e uma de definições (versão do codebook, título, tipo, descrição, ordem).
- A definição pertence a uma **versão** do codebook, não ao projeto. Salvar copia o conjunto inteiro para a versão nova. Isso torna cada versão um retrato imutável, que é o que permite amarrar avaliações futuras a um ID de versão.
- `descricao` da definição nasce nula. Ela é preenchida na Fase 2, conforme a ADR 0001.
- O tipo da definição usa o mesmo vocabulário do glossário: Categoria, Dimensão de qualidade e Diretriz. O tipo de tarefa do projeto pré-seleciona o valor, sem mudar comportamento nenhum, conforme a ADR 0005.
- Numeração sequencial calculada como maior versão existente mais um, dentro da mesma transação que insere, com restrição de unicidade em (projeto, número) para garantir o resultado sob concorrência.
- Regra de salvamento nesta fase: pelo menos uma definição. A regra "cada definição precisa de ao menos um critério" pertence ao Épico 2, junto com os critérios.
- A versão ganha a coluna `usada_em` (timestamp, nula). Ela nasce nula e será preenchida no Épico 2, na mesma transação que cria a rodada que consome a versão. Uma versão está **em aberto** quando `usada_em` é nula **e** ela é a de maior número do projeto; nos outros casos está congelada.
- Guardar o congelamento como coluna, em vez de derivar de uma tabela de rodadas que ainda não existe, é o que deixa a regra implementável e testável já neste épico: o teste preenche `usada_em` à mão e prova que a edição é recusada.
- Salvar numa versão em aberto substitui o conjunto inteiro de definições daquela versão, do mesmo jeito que criar versão nova o copia. Não há edição parcial de definição solta.
- Salvar numa versão congelada cria a versão seguinte, copiando o conteúdo e aplicando as mudanças. A decisão entre atualizar e criar é do servidor, não de um botão diferente na tela.

Prompt

- Tabela de versões de prompt (projeto, número da versão, texto, nome, descrição, registro de mudanças, autor, data), versionada de forma independente do codebook, conforme a ADR 0002.
- Mesma coluna `usada_em` e mesma noção de versão em aberto do codebook, com a mesma função de checagem. Alterar o texto de uma versão em aberto é edição no lugar; alterar o texto de uma versão congelada cria a versão seguinte.
- Alterar nome, descrição ou registro de mudanças é sempre edição no lugar, permitida na versão mais recente esteja ela em aberto ou congelada. Metadado não entra no envio à LLM, então não tem por que gerar versão nem por que congelar.
- "Testar o prompt" não preenche `usada_em`, porque não persiste resposta nenhuma. Quem preenche é a rodada do Épico 2.

Itens de entrada

- Tabela de itens ligada ao **projeto**, não à rodada nem à fase. O glossário já diz que os itens vivem num pool do projeto e que as fases amostram dele.
- Conteúdo guardado em coluna de texto. O upload é conveniência de digitação: o arquivo é lido no navegador e o texto vai para o campo. Não há Supabase Storage envolvido, e o arquivo original não é guardado nem fica disponível para download.
- Formatos aceitos por lista de permissão, numa constante única compartilhada entre o atributo `accept` do seletor e a validação: `.txt`, `.md`, `.csv`, `.tsv`, `.json`, `.xml`, `.html`, `.yaml`, `.log` e extensões de código. Como o critério real é "decodifica como texto", a validação final é a própria leitura: se o resultado não for texto legível, o arquivo é recusado.
- PDF e DOCX ficam de fora porque exigiriam biblioteca de extração no servidor, e extração de PDF é um problema com casos ruins de sobra (PDF escaneado, coluna dupla, tabela) que não pertencem a este épico. A saída é exportar como texto, e a mensagem de recusa diz isso.
- Dois limites, ambos constantes em `lib/limits.ts`: `ITEM_CONTENT_MAX` de 50.000 caracteres, espelhado numa CHECK conforme o padrão que o próprio arquivo documenta, e `ITEM_FILE_BYTES_MAX` de 2 MB, conferido no navegador antes de ler o arquivo. O primeiro é a regra; o segundo é só para ninguém travar a aba tentando abrir um arquivo absurdo.
- Não há partição de itens entre treino e teste. A palavra "treino" não pertence a este domínio, porque nada é treinado. Evitar reuso de item na Fase 4 é responsabilidade humana, e a ferramenta ajuda mostrando em quais rodadas cada item já foi usado, sem filtrar e sem travar.

Integração com a LLM

- Provedor: OpenAI, com API paga. O identificador do modelo vem de variável de ambiente e precisa ser conferido na documentação da OpenAI no momento de implementar, porque esses nomes mudam.
- Módulo novo `lib/ai`, marcado como `server-only`, expondo uma função só: recebe o texto composto e devolve o texto da resposta mais os metadados do modelo. A chave nunca aparece em variável `NEXT_PUBLIC_`.
- Composição do envio, conforme a ADR 0002: nas Fases 1 e 2 vai o texto do prompt mais os títulos das definições; nas Fases 3 e 4 vai o codebook completo. Os critérios nunca vão à LLM antes da Fase 3.
- A saída é texto livre e opaco para a ferramenta. Não há parsing de JSON nem formato imposto. Se o Administrador quiser saída estruturada, ele pede isso no texto do prompt, que é livre.
- "Testar o prompt" chama `lib/ai` e devolve o texto para a tela sem gravar nada: sem Resposta, sem Rodada, sem proveniência. A ADR 0001 muda nesse ponto, porque ela previa um smoke test que persistia poucas respostas.

- Teto de respostas por projeto configurável por variável de ambiente, para que um erro em épico futuro não vire fatura.

Administrador como avaliador

- A restrição de unicidade de `project_members` já é por (projeto, usuário, papel), então a mesma pessoa pode ter duas linhas, uma de administrador e outra de avaliador. Não há mudança de schema.
- A linha de avaliador não é isenta de consentimento nem de questionário, ao contrário da linha de administrador. O Administrador que se dá acesso entra pelo mesmo fluxo de onboarding que já existe.
- A ação é reversível enquanto não houver avaliação submetida por aquele vínculo. Depois disso ela trava, porque avaliação enviada é imutável.
- Os painéis de concordância do Épico 2 precisam conseguir separar o administrador que avalia dos demais avaliadores. Quem escreve o codebook avaliando com ele infla a concordância, e essa limitação precisa aparecer na análise em vez de ficar escondida.

Autorização

- Tudo neste épico é ação de Administrador de Projeto e é checado na camada de aplicação, via `lib/authz`, conforme a ADR 0007. Não há RLS, trigger nem RPC. O documento antigo pedia validação estrita de papel no backend, e é isso que a checagem explícita entrega.
- O requisito do documento antigo que pedia bloqueio de imutabilidade "no banco de dados" é reescrito: imutabilidade vive na camada de aplicação, apoiada em restrições declarativas de schema. Do jeito que estava escrito, ele reabria a ADR 0007 sem dizer.

Decisões de modelagem que este épico registra mas não constrói

Ficam escritas aqui porque nascem das decisões acima e o Épico 2 vai depender delas:

- Toda Resposta grava proveniência: origem (gerada pela ferramenta ou colada à mão), modelo, versão do modelo, versão do prompt e versão do codebook. É o que a ADR 0002 exige para replicabilidade.
- A Rodada existe da Fase 2 em diante e aponta para o par de versões (prompt e codebook) mais o conjunto de itens que consumiu, através de uma tabela de ligação.
- Geração processa de 1 a 5 **itens** por vez, uma resposta por item, sem repetir o mesmo item. Ler o número como "respostas por item" misturaria a variação não determinística do modelo com a discordância entre avaliadores, que é justamente o que a Fase 2 mede.
- A avaliação se vincula ao vínculo de membro, não ao usuário, para preservar com qual papel a pessoa avaliou. Remover um avaliador passa a significar desativar, porque apagar membro não pode apagar dado de pesquisa.

- Painéis de discordância e anotações de consenso são escopados à rodada. O avaliador só enxerga os da rodada em que está avaliando, senão a Fase 4 mede a memória do grupo em vez da clareza do codebook.

Fora do Escopo

Realocado para o **Épico 2 (Fase 2)**, com o texto das histórias aproveitado do documento anterior:

- Critérios de avaliação, específicos e gerais, e a herança automática dos gerais.
- Descrição das definições, que é o segundo tempo do codebook.
- Geração de respostas em lote e persistência de Resposta.
- Telas de avaliação, notas e justificativas.
- Painel de discordância e anotações de consenso.
- Entidades Rodada e Resposta, com a proveniência descrita acima.
- Cálculo de concordância entre avaliadores.

Fora do escopo do projeto por decisões anteriores que continuam valendo:

- Chave de API trazida pelo usuário, escolha de outro modelo ou de outra LLM e limites de requisição sobre a chave da ferramenta, todos adiados pela ADR 0003.
- Colar resposta manualmente, prevista na ADR 0003 e ainda sem épico.
- LLM como avaliadora, que segue como item em aberto no glossário.
- Entrada em projeto por código de sala. O acesso continua sendo por convite por e-mail, conforme a ADR 0006.
- Partição de itens entre conjuntos reservados, e qualquer trava automática de reuso de item.
- Extração de texto de PDF, DOCX e outros formatos binários, e OCR de documento escaneado. Entram quando houver demanda real, e provavelmente com um épico próprio, porque a qualidade da extração vira parte do dado de pesquisa.
- Guardar o arquivo original, oferecer download dele ou qualquer gestão de arquivo. O que o projeto guarda é o texto.
- Reabrir versão congelada, comparar duas versões lado a lado e restaurar versão antiga como nova.